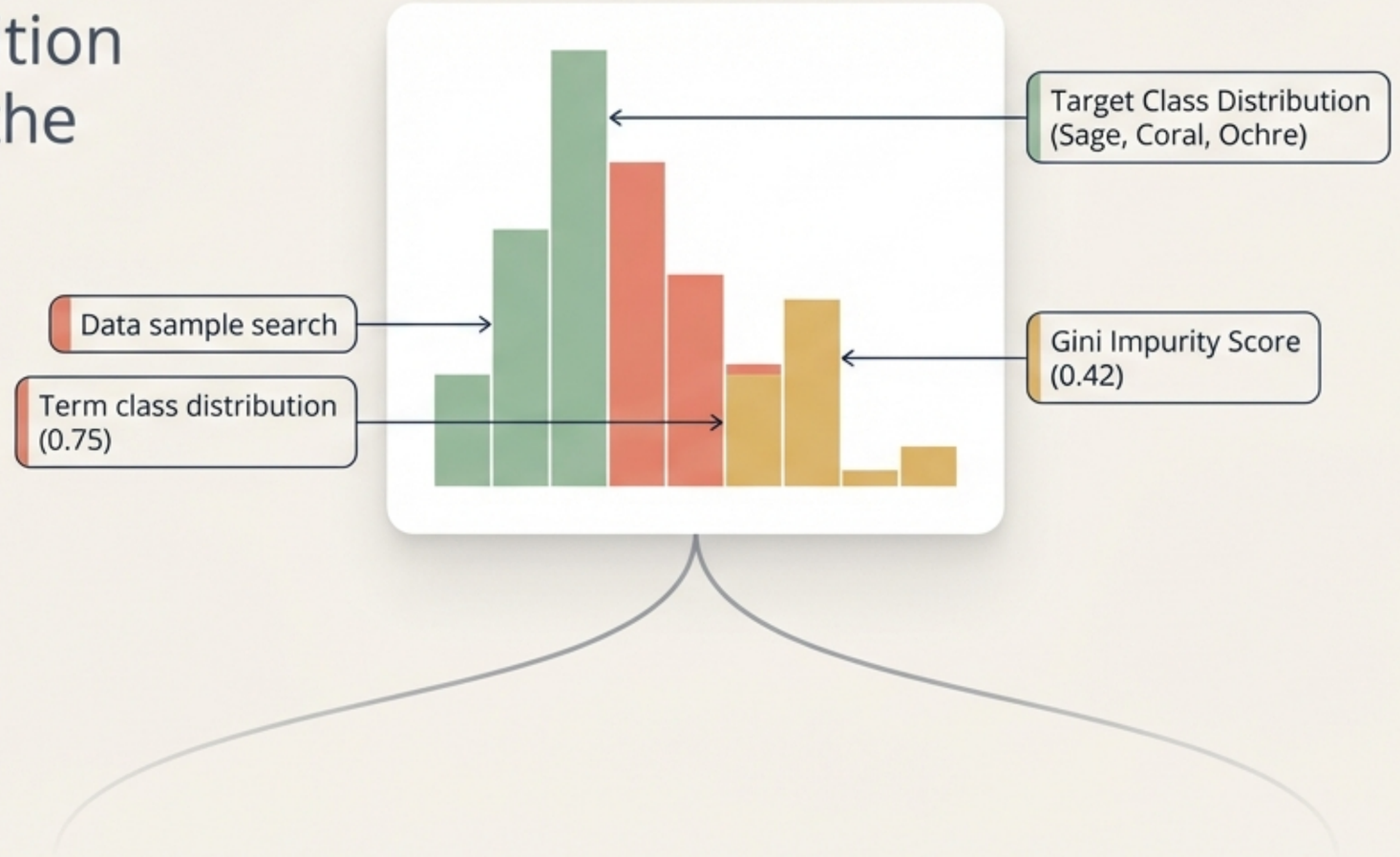
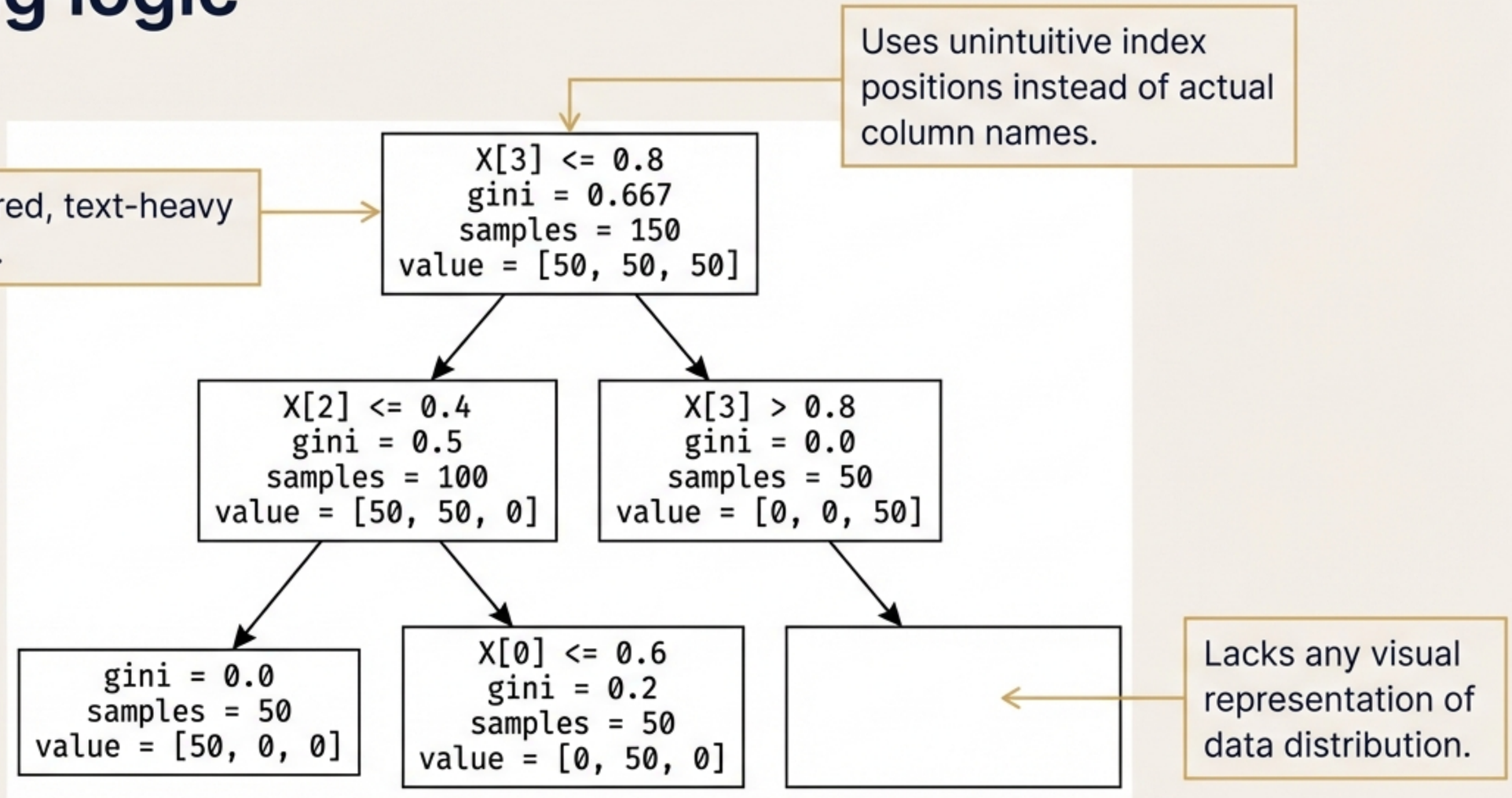
Mastering Decision Tree Visualisation

Unlocking true model intuition and interpretability using the dtreeviz Python library.



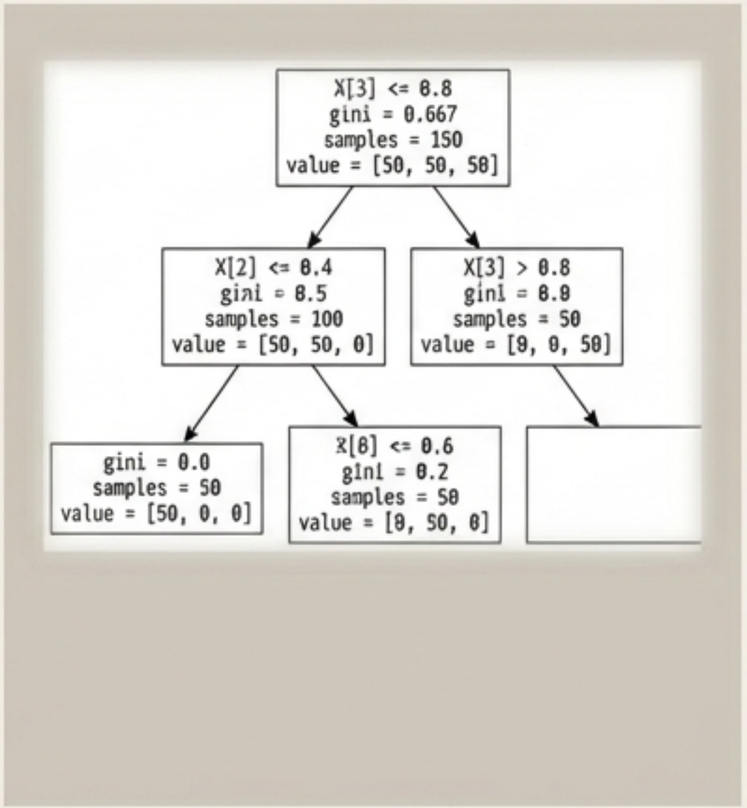
Standard decision tree plots obscure the underlying logic



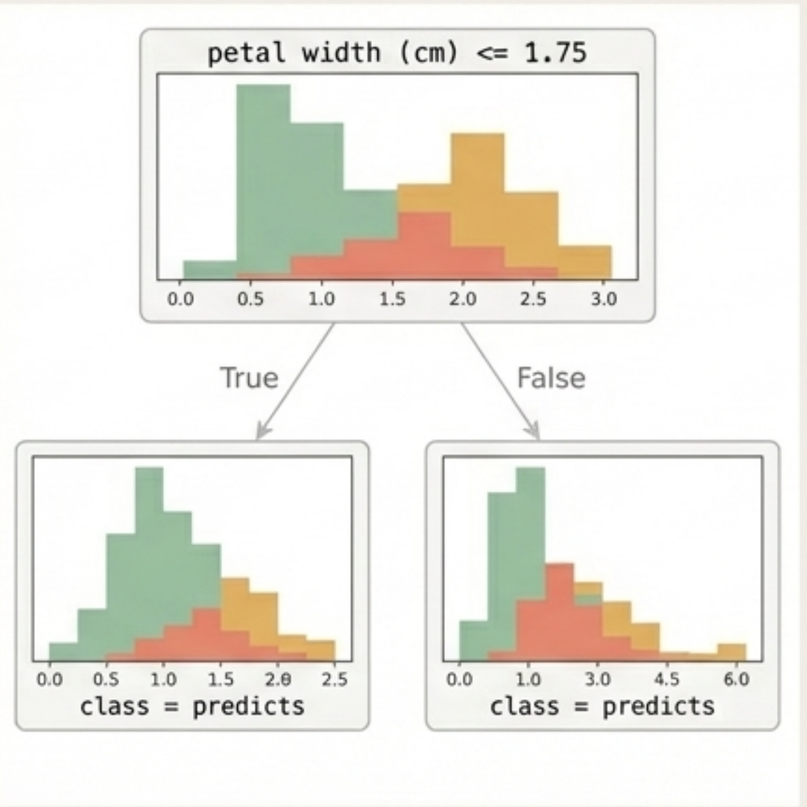
Data intuition requires clear, data-rich visualisations

- Beautiful, presentation-ready aesthetics.
- Displays exact splitting criteria mapped to real feature names.
- Visualises actual training data distributions directly inside the nodes.

Before



After



Integrating dtreeviz into your workflow

```
pip install dtreeviz
```

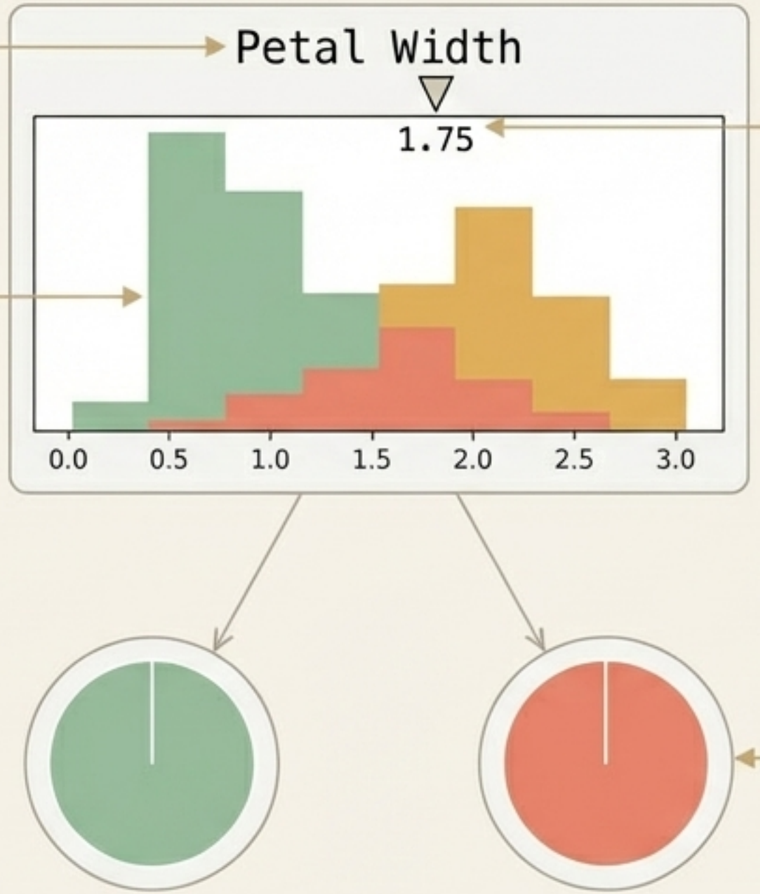
```
import graphviz
from IPython.display import Image, display_svg
```

Note: Graphviz and IPython dependencies are essential for rendering interactive plots directly within Jupyter environments.

The anatomy of a classification tree node

Feature Name: Clearly displays the target column.

Split Value: Shows the exact numerical boundary.

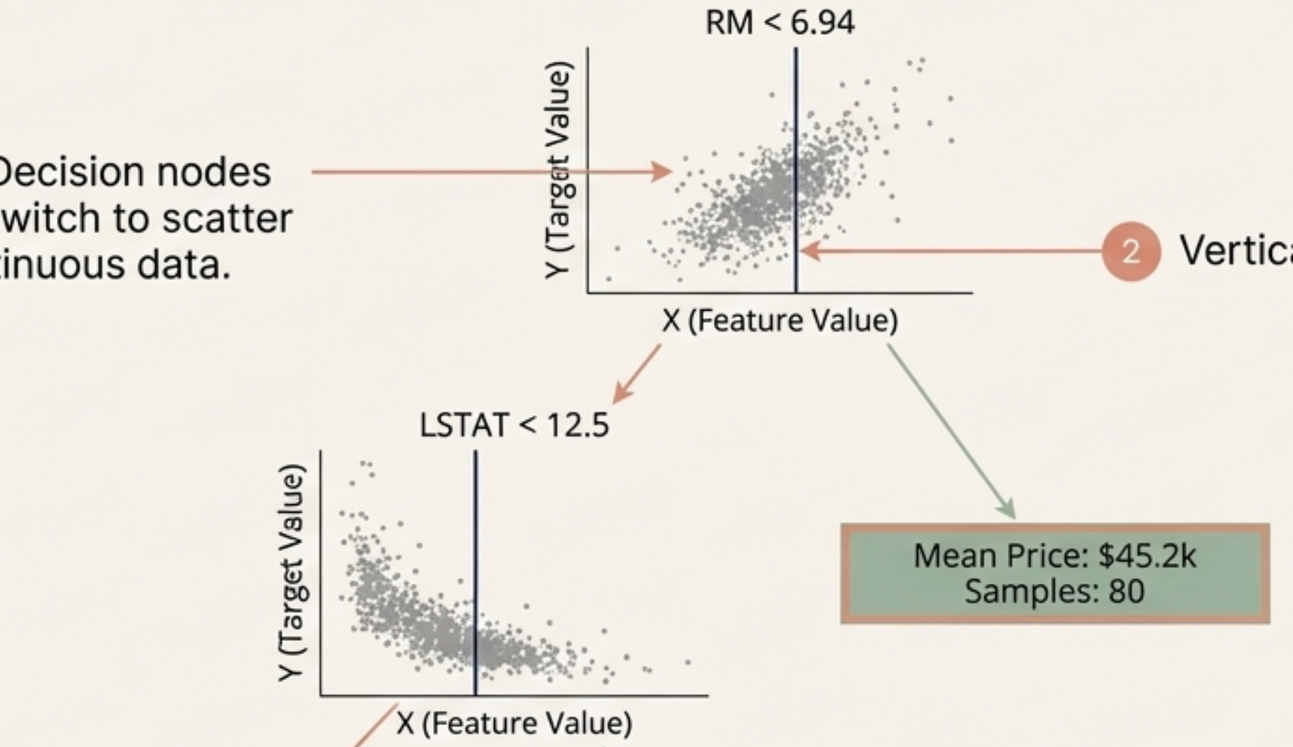


Data Distribution: Stacked histogram reveals how distinct classes are distributed.

Pure Leaf Nodes: Terminal nodes show a pure class majority.

Adapting visual language for continuous targets

1 Visual Shift: Decision nodes intelligently switch to scatter plots for continuous data.



2 Vertical split lines cut directly through the continuous data.

3 Leaf nodes display the calculated mean price and total sample size for that branch.

Tracing the exact prediction path of a single observation

```
dtreeviz(..., X=new_data_point)
```

Pass a single array (X) into the visualizer to instantly map exactly why a specific prediction was made.

Translating decision logic into plain English

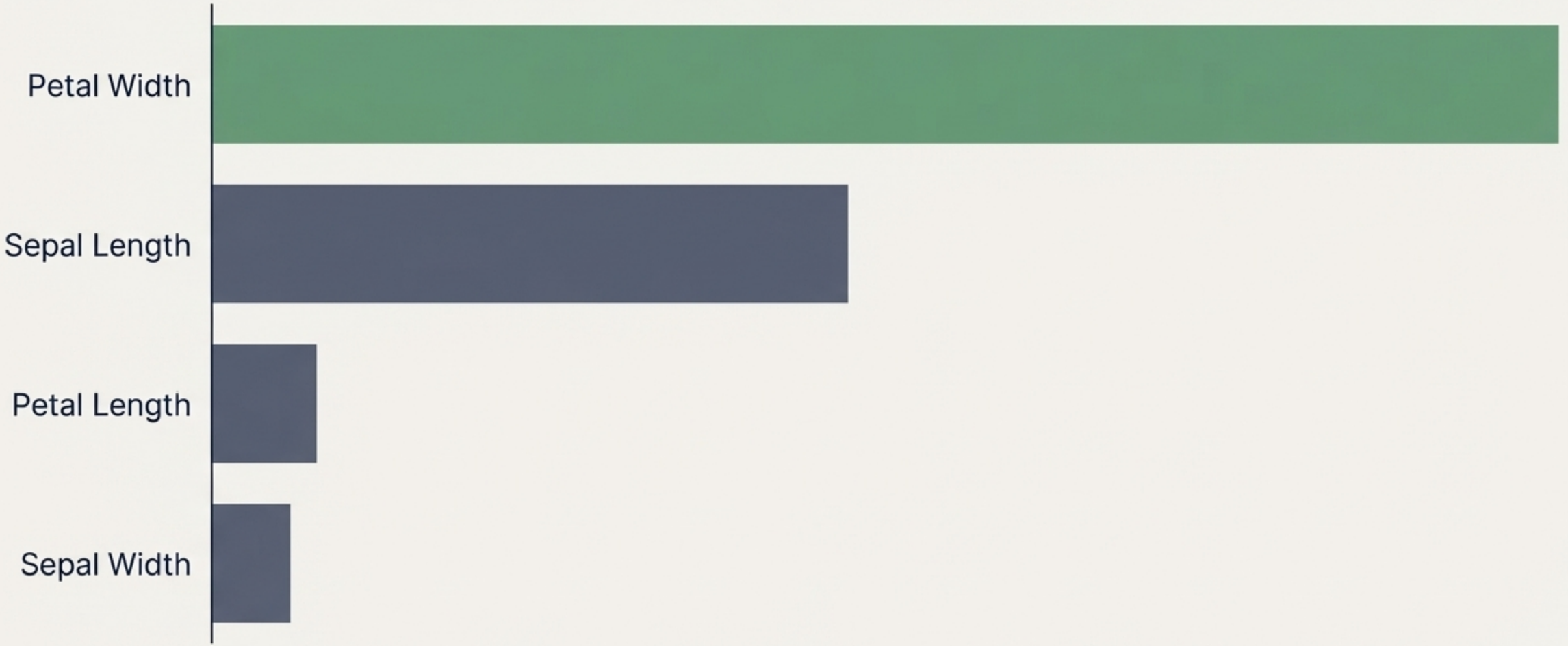
Perfect for extracting readable rules and translating model logic to non-technical teams and stakeholders.

```
1.5 ⇐ Petal Length
Petal Width < 1.75

Prediction: Versicolor
```

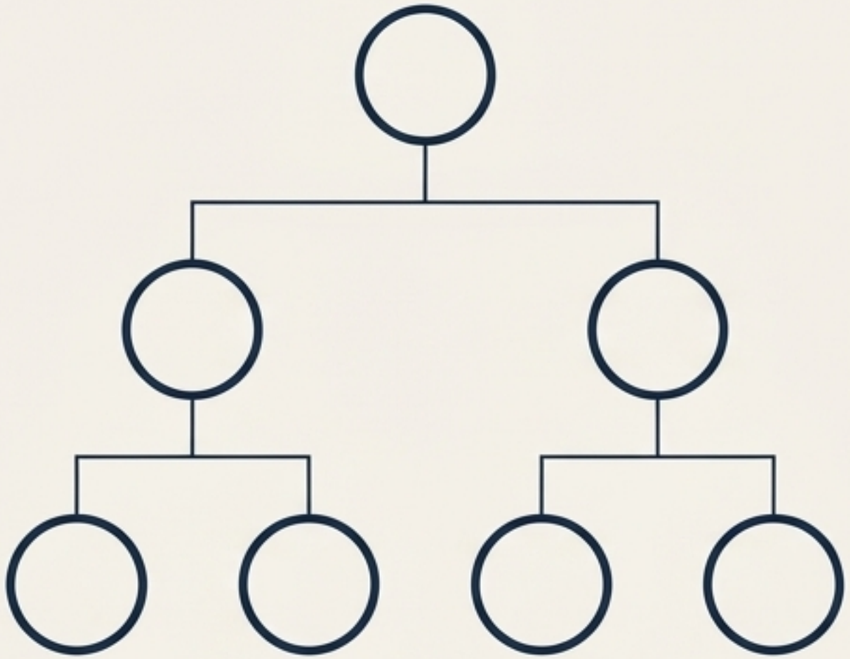
Visualising global feature importance

Beyond individual node splits, the library provides a clean, ranked visual of the exact features driving the model's overall decisions.



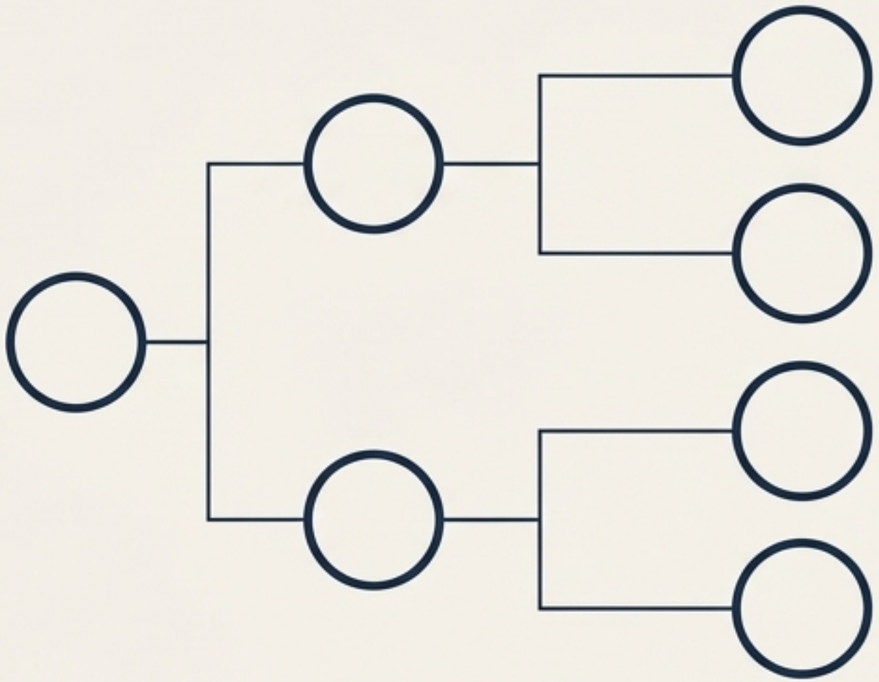
Taming massive and highly complex trees

Powerful configuration options to manage large-scale and intricate models.



`fancy=False`

Removes internal charts to purely show the mathematical structure.

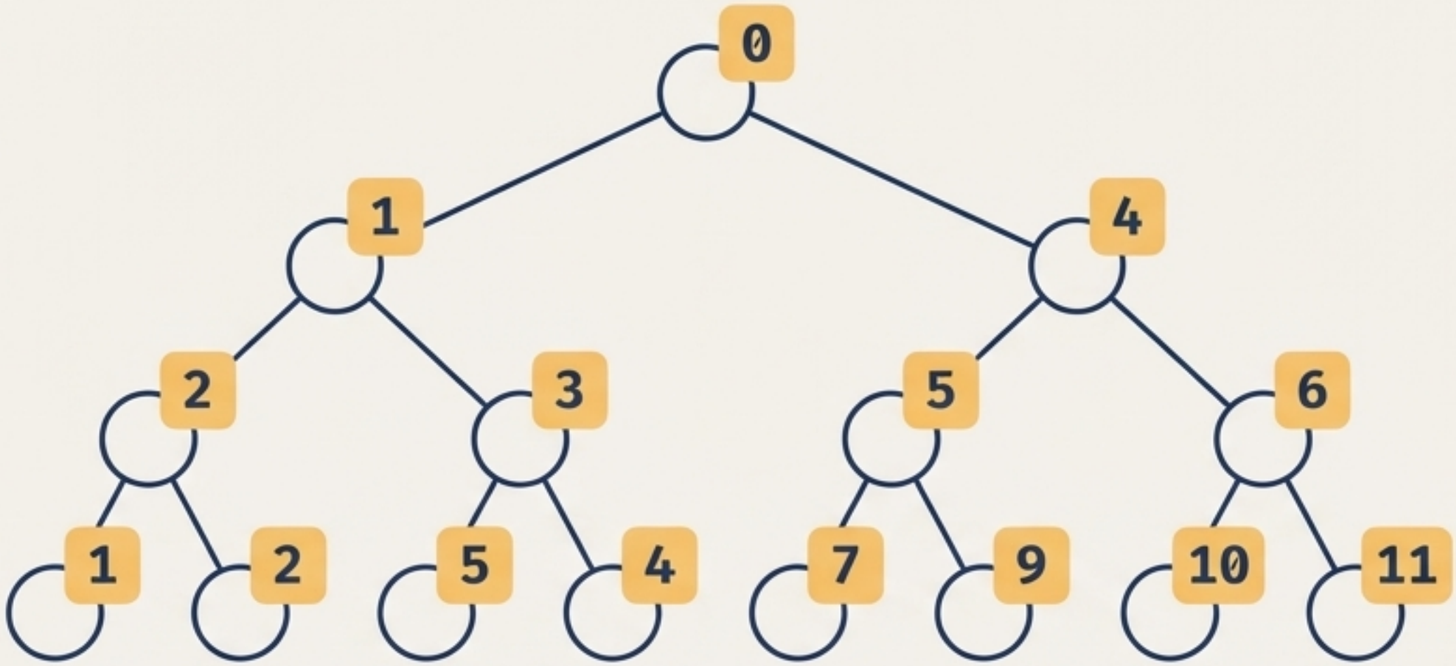


`orientation='LR'`

Shifts the tree to a Left-to-Right orientation, ideal for deep models and scrolling.

Mapping the depth-first node traversal

`show_node_labels=True`

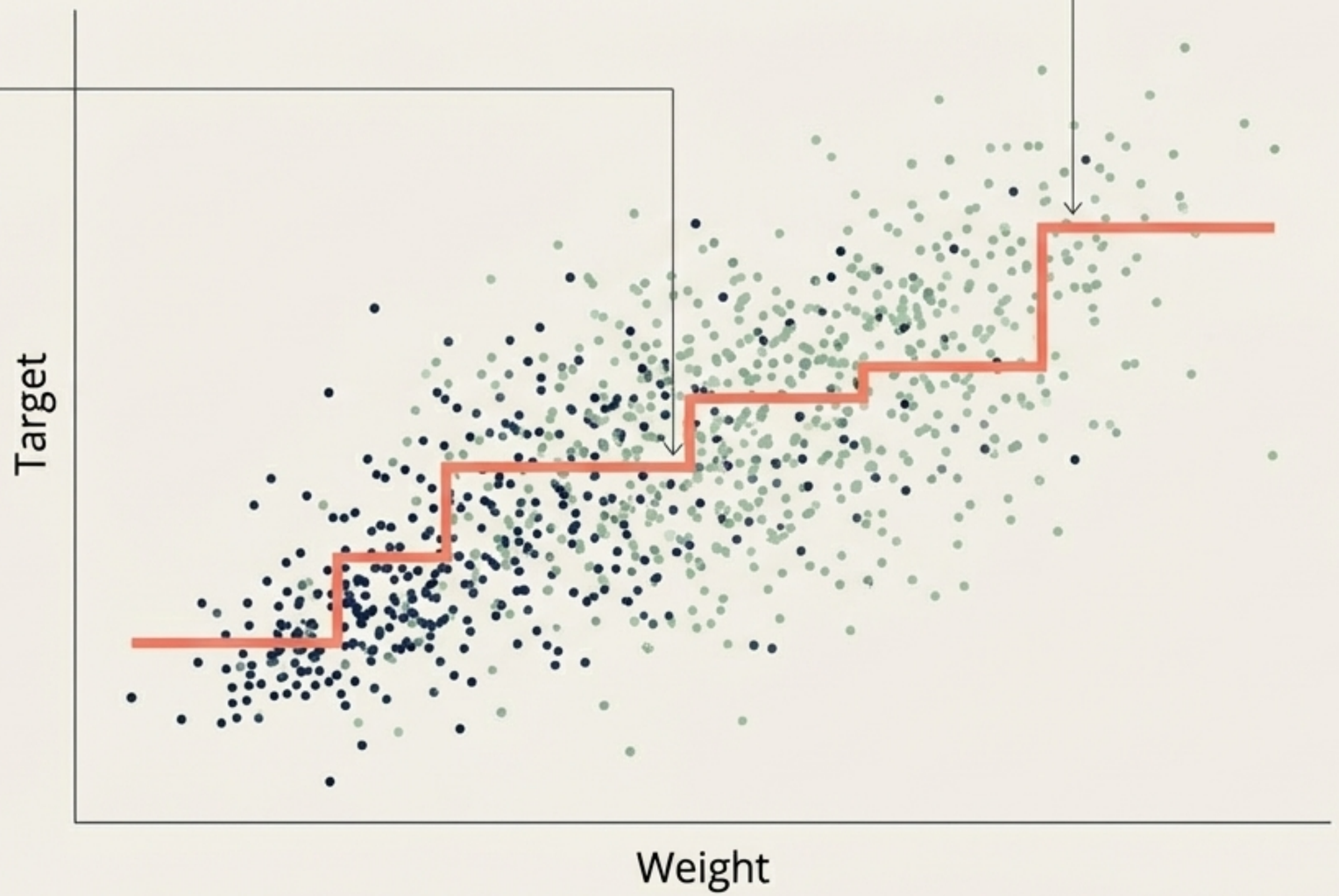


Reveals the exact depth-first programmatic order used by the algorithm—crucial for debugging and mapping splits directly back to code.

Mapping univariate decision boundaries

`rtreeviz_univar(...)`

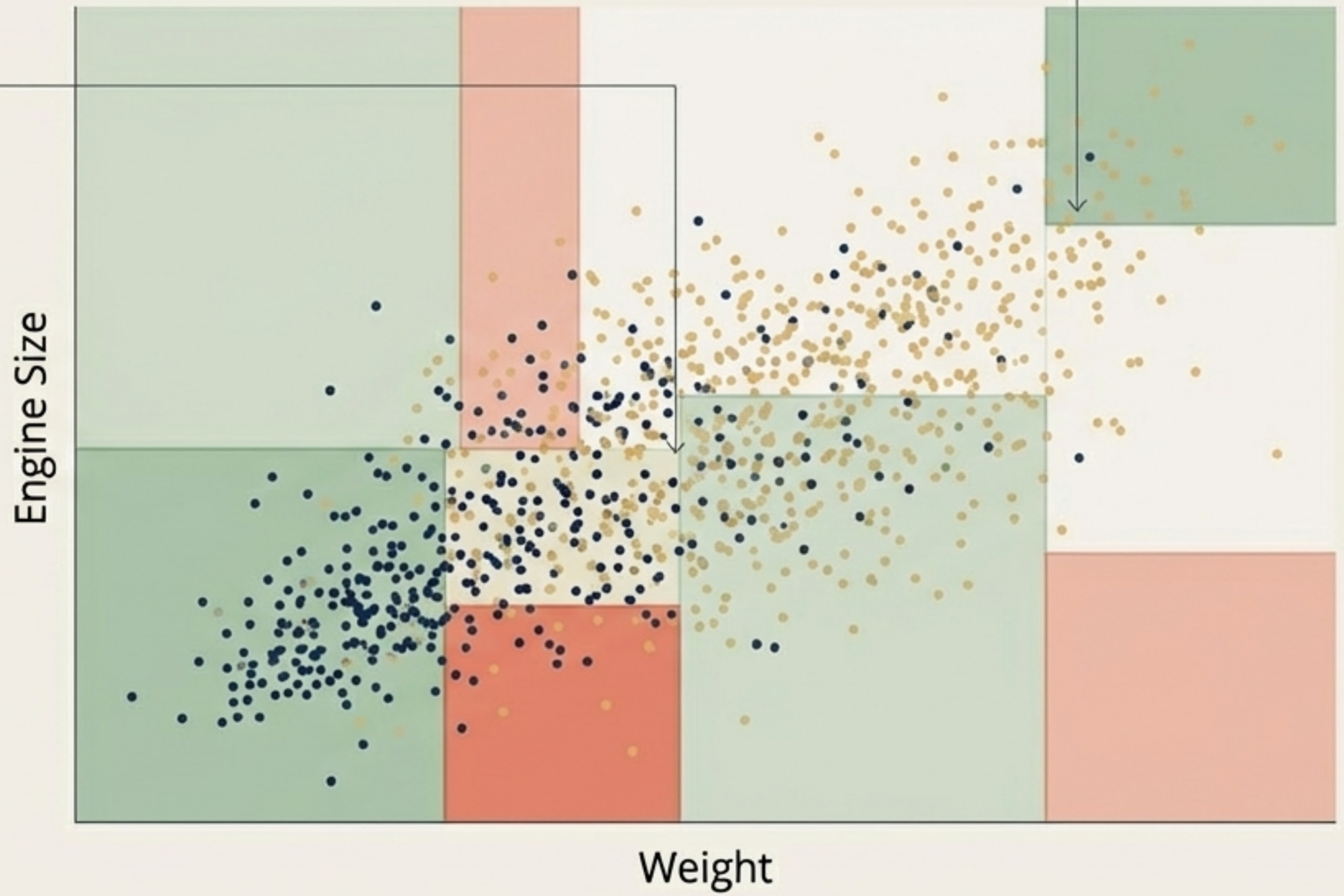
Overlay the tree's step-function directly onto the dataset to observe precisely where the algorithm slices continuous data.



Slicing the 2D feature space

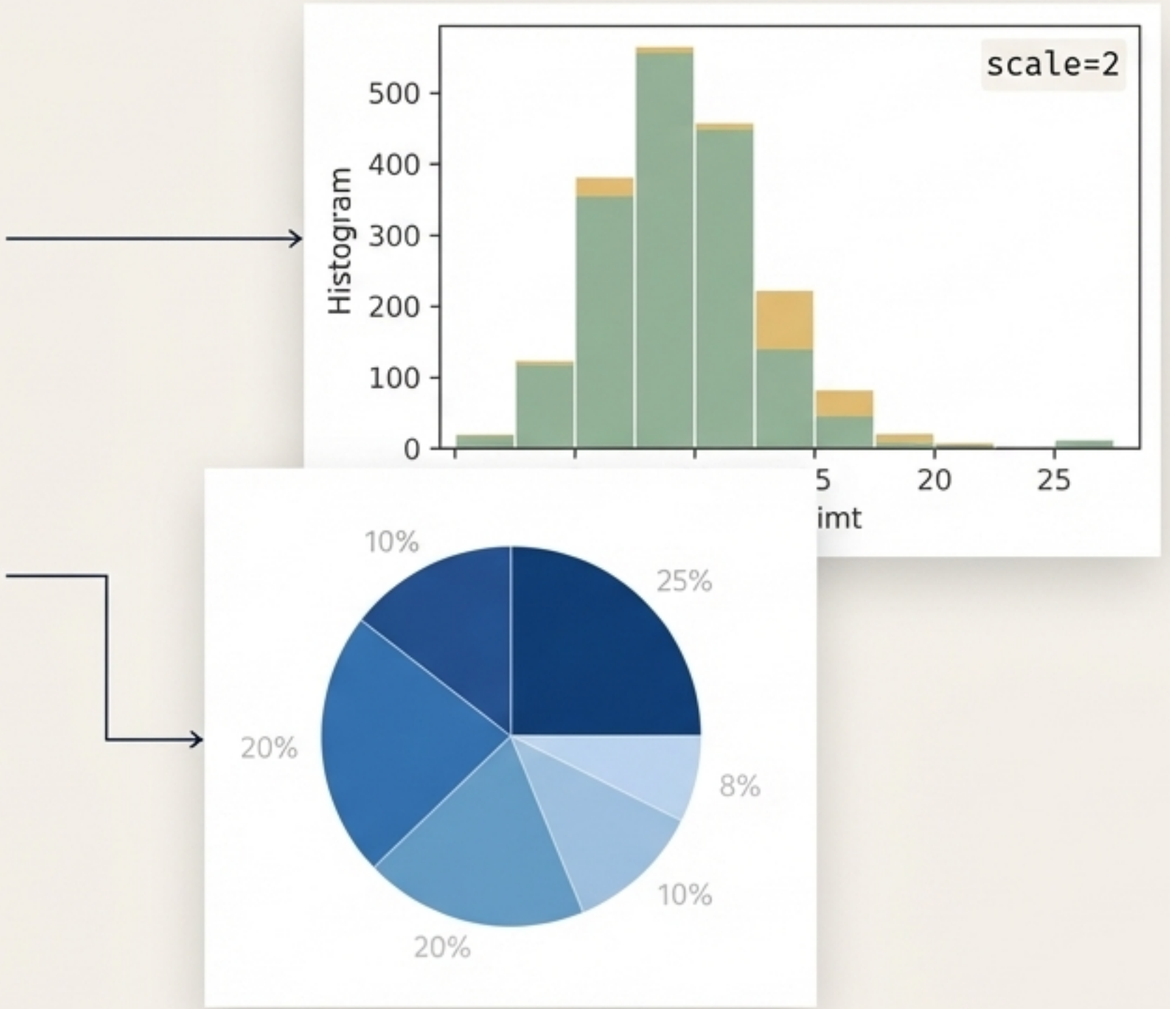
`rtreeviz_bivar(...)`

Visualise how the algorithm segments a two-dimensional feature space into discrete, actionable regions.



Customising outputs for reports and presentations

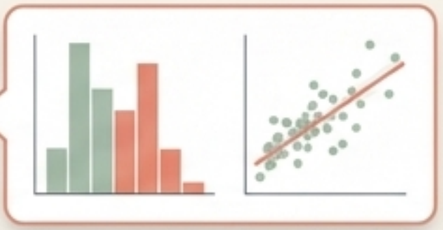
- Use the `scale` parameter (e.g., `scale=2`) to generate high-resolution, oversized outputs.
- Easily modify fonts, colours, and chart styling to match corporate branding.
- Check the official GitHub repository for advanced CSS-style tweaking.



Elevating model interpretability

Ditch legacy plots: Replace cluttered, index-based visuals with data-rich node visualisations.

Tailor the view: Intelligently adapt charts for both **classification** (histograms) and **regression** (scatter plots).



Enhance communication: Use path tracing and English translations to explain **exact model logic** to any stakeholder.